

Python Question

Q1. What is python?

Python is a high-level interpreter programming language which is created by Guido Van Rossum in 1991. The idea behind the python language to create a high-level programming language that can be use in verity of application and also easy to use and understand python have a very easy syntax that can be even read by humans it is base on the OOP's programming concept that make a python ideal for creating apps, data analysis, and AI/ML applications.

Q2. What are the features of python?

Python is a feature rich programming language:

- It is interpreting programming language.
- Python is a Open Source language
- Python have most verity of open-source libraries
- It is a platform in depended language
- It has a global community support.
- Python also have a GUI support

Q3. What are the applications of python?

Python is a feature rich language with an open source license due to this very large number of developer contribute to this language worldwide because of this we use python in verity of task:

- Data Science & Analytic
- Game Development
- Web Development
- GUI creation
- Mathematics and statistics
- Software development

Q4. What is variable?

A variable in programming is like a name we give to a place in memory where we keep some data. The value we store can change while the program runs It's kind of like a box with a label on it — you can put something inside it (like a number or some text) and later open it to check or change what's inside.

For Example: `a = 5`

```
print(a)
```

output : 5 this example show that we store 5 in the a variable a .

Q5. What is datatype?

A data type tell computer what kind of values is store or will be store in the variable.

Data types helps computer differentiating the value of a variable and helps it how to use it

Types of Datatype:

1. **Int:** is use to store numeric values like 1, 5 etc.
2. **Float:** is use to store decimal values like 2.5, 4.56 etc.
3. **String(str):** is use to store Text like “Hello World”.
4. **Bool:** is use to store boolien values “True or False ” or “0 or 1”.

Q6. What are the identifiers?

Identifiers are the names we give to different parts of a program, like variables, functions, or classes. They make it easy to talk to the computer and remember what each thing does.

For example, if we create a variable to store someone’s age, we can name it age. Then whenever we use age in the program, the computer knows we’re talking about that specific variable.

For Example:

Name = ‘Omkar’

Age = 21

Key note :

Identifiers are case sensitive so Name and name are two different identifier.

Q7. What is the difference between mutable & immutable datatype?

No	Mutable Data Type	Immutable Data Type
1	Values can be changed after creating them.	Values cannot be changed once created.
2	When you modify it, the same memory is used.	When you modify it, a new memory is created.
3	Examples: list, dict, set	Examples: int, float, str, tuple

Q8. What is the Difference between list & tuple?

In Python, lists and tuples are both used to store collections of items, but they differ in how we can use and modify their contents.

List

- A list is **mutable** — you can change, add, or remove elements.

- It is created using **square brackets []**.
- Lists are **slower** compared to tuples.
- Used when you want data that **can change** during the program.
- It **takes more memory**.
- You can use many built-in methods like `append()`, `remove()`, `pop()`, etc.
- Good for tasks where you need to **update or modify** data frequently.
- `fruits = ["apple", "banana", "mango"]`

Tuple

- A tuple is **immutable** — you can't change or update its elements once created.
- It is created using **round brackets ()**.
- Tuples are **faster** in performance.
- Used when you want data that **should not change**.
- It **takes less memory**.
- It has fewer built-in methods since it can't be changed.
- Good for **fixed collections of data**, like coordinates or constant values.
- `colors = ("red", "green", "blue")`

Q9. What is the Difference between list & set?

In Python, both lists and sets are ways to store multiple items, but they serve different purposes depending on what you need in your program

No	List	Set
1	A list is always in an order format	In a set order does not matter it can have unique item in it
2	List is mutable we can add, remove, or change items easily.	Set is also mutable we can add or remove items, but elements themselves must be immutable
3	In List Duplicates are allowed: we can write "apple" twice if we want	In Set Duplicates are automatically removed: You can't have two of the same items
4	When we care about the order of items or need repeated values.	When we need unique values, like IDs, tags, or to check membership quickly.
5	Syntax: Square brackets []	Syntax: Square brackets {}.
6	Ex. <code>fruits = ["apple", "banana", "apple"]</code>	Ex. <code>fruits_set = {"apple", "banana", "apple"}</code>

Q10. What is dictionary?

A **dictionary** in Python is a way to store data in **key-value pairs**, kind of like a real-life dictionary where a word (key) points to its meaning (value).

- Each **key** is unique, and it is used to access its corresponding **value**.
- **Values** can be anything — numbers, text, lists, or even another dictionary.
- Dictionaries are **mutable**, so you can add, remove, or change items.
- They are **unordered** in older Python versions (<3.7), but in Python 3.7+, they **remember the order** items were added.

Example:

```
student = {  
    "name": "Aarav",  
    "age": 21,  
    "course": "Data Analytics"  
}  
  
print(student["name"]) # Output: Aarav  
  
# Adding a new key-value  
student["grade"] = "A"  
  
# Updating a value  
student["age"] = 22
```

Think of a dictionary like a **box of labelled compartments** — the label (key) tells you exactly where to find the item (value).

Q11. What is Difference Between split & join method of string?

The split() Method

- **Action:** You use split() on a string.
- **Purpose:** It **breaks** that single string into an **array** (or list) of smaller strings.
- **How:** It uses a separator (or "delimiter") you provide to know where to make the cuts.

Ex.

```
text = "apple,banana,orange"  
  
result = text.split(",")  
  
print(result) # Output: ['apple', 'banana', 'orange']
```

The join() Method

- **Action:** You use `join()` **on an array** of strings.
- **Purpose:** It **combines** (or 'joins') all the items from that array into **one single string**.
- **How:** It uses a separator (or "glue") you provide to place *between* each item as it builds the new string

Ex.

```
fruits = ["apple", "banana", "orange"]
result = " - ".join(fruits)
print(result) # Output: "apple - banana - orange"
```

Q12. Difference between up and extend in List

The key difference is that **`append()`** adds **one single element** to the end of a list, while **`extend()`** adds **all the elements** from an iterable (like another list) to the end.

Here's the simplest way to think about it:

- If I have a list, say `my_list = [1, 2, 3]`.
 - And I also have `new_items = [4, 5]`.
1. If I use **`append()`**, like `my_list.append(new_items)`, the *entire* `new_items` list is added as one single element. The result would be a nested list: `[1, 2, 3, [4, 5]]`.
 2. But if I use **`extend()`**, like `my_list.extend(new_items)`, it unpacks `new_items` and adds each item individually. The result would be one flat list: `[1, 2, 3, 4, 5]`.

So, you use `append` when you want to add one item (even if that item is another list), and you use `extend` when you want to merge a list with another list or iterable."

Q13. What is set default method does in dictionary?

Certainly. The `setdefault()` method is a way to **get a key's value, while simultaneously setting a default value if that key doesn't exist**.

It follows a simple two-step logic:

1. **If the key already exists:** It just **returns** the key's current value. (It does *not* overwrite it).
2. **If the key does *not* exist:** It **inserts** the key with the default value you provided, and then **returns** that new default value.

It's essentially a "get-or-set-if-missing" operation in one step.

Simple Example

Let's start with a dictionary:

```
my_dict = {'a': 1}
```

Case 1: Key 'a' EXISTS

```
value = my_dict.setdefault('a', 99)
```

```
print(value)    # Output: 1 (It returned the existing value)
```

```
print(my_dict) # Output: {'a': 1} (The dictionary is unchanged)
```

Case 2: Key 'b' does NOT EXIST

```
value = my_dict.setdefault('b', 0)
```

```
print(value)    # Output: 0 (It returned the new default value)
```

```
print(my_dict) # Output: {'a': 1, 'b': 0} (The dictionary was modified)
```

Q14. What is function?

In Python, a **function** is a reusable block of code that performs a specific task.

Think of it as a recipe: you define it once (the recipe) and then you can "call" it (cook the recipe) as many times as you want without having to rewrite the instructions.

The main reasons to use functions are:

- **Reusability:** Write code once and use it multiple times.
- **Organization:** Break down complex programs into smaller, manageable, and more readable parts.

How to Define and Call a Function

You use the `def` keyword to create a function and give it a name. You "call" the function by using its name with parentheses `()`.

```
# 1. DEFINING the function
```

```
def greet():
```

```
    print("Hello, world!")
```

```
# 2. CALLING the function
```

```
greet()
```

```
greet()
```

Output:

```
Hello, world!
```

```
Hello, world!
```

Q15. What are the type of function?

Main Types of Functions in Python

Functions in Python can be grouped into three main categories:

1. **Built-in Functions:** These are functions that are built directly into Python and are always available. You don't need to import or define them.
 - **Examples:** `print()`, `len()`, `type()`, `int()`, `input()`
2. **User-Defined Functions (UDFs):** These are the functions that you, the programmer, create using the `def` keyword. The `add_numbers` function above is a perfect example. They perform custom tasks specific to your program's logic.
3. **Anonymous Functions (Lambda Functions):** These are small, one-line functions that do not have a name. They are created using the `lambda` keyword. They are typically used for simple operations, especially as an argument to another function (like `sorted()`, `map()`, or `filter()`).
 - **Example:** A lambda function that doubles a number:

```
double = lambda x: x * 2
```

```
print(double(5)) # Output: 10
```

Q16. What is Difference between user defined function & lambda function?

The primary difference is that a **user-defined function**, made with the `def` keyword, is a **named** block of code for complex, reusable tasks, while a **lambda function** is an **anonymous**, single-line function for simple, immediate operations.

Here's how I think about them in my data analytics work:

User-Defined Function (def)

I use a `def` function for any significant, repeatable logic.

- **Named and Reusable:** It has a name (like `def clean_text_data(...)`) which means I can call it multiple times, in different places, and my code is more readable.

- **Multi-Line and Complex:** This is the main point. I can use multiple lines, loops, if-else logic, error handling, and multiple return statements. This is essential for building preprocessing pipelines or defining custom evaluation metrics.
- **Documentation:** It supports a **docstring**, which is critical for making my code maintainable and understandable for my teammates.

For example, I would *always* use a `def` function to define a feature engineering step:

```
def get_age_group(age):
    """Categorizes an age into predefined groups."""
    if age < 18:
        return 'Under_18'
    elif 18 <= age <= 65:
        return 'Adult'
    else:
        return 'Senior'
```

Lambda Function (lambda)

I think of lambda functions as 'throwaway' or 'helper' functions.

- **Anonymous:** It has no name. You define it right at the moment you need to use it.
- **Single Expression:** This is its main limitation. A lambda can only contain *one single expression*. It can't have loops, if-else (other than a ternary operator), or explicit return statements. It just returns the result of that one expression.
- **Data Science Use Case:** In my work, the most common place I use lambda is directly inside a **Pandas .apply() or .map() method**.

It's perfect for a simple, one-time data transformation. For instance, if I just need to quickly square a column:

```
# No need for a full 'def' function for this simple task
df['col_squared'] = df['col'].apply(lambda x: x**2)
```

Q17. What is anonymous function?

An anonymous function is simply a function without a name. They are often called **lambda functions** in Python or **arrow functions** in JavaScript.

Instead of defining a function with `def` and then calling it, you define it "inline," right where you need it.

What is their main purpose?

Their main purpose is to be passed as an argument to **higher-order functions**—functions that take other functions as input. This is very common for operations like sorting, mapping, or filtering.

For example, if you have a list of products and you want to sort them by price, you can use a lambda function for the sorting key:

```
# Sort a list of (product, price) tuples by price
```

```
products = [('apple', 2.99), ('banana', 0.99)]
```

```
# The lambda function provides the sorting logic on the fly
```

```
sorted_products = sorted(products, key=lambda item: item[1])
```

```
# sorted_products is now [('banana', 0.99), ('apple', 2.99)]
```

The lambda `item: item[1]` is the anonymous function. It's a short, temporary function that just says, "use the second element for sorting."

Q18. What is Recursion

Recursion is a programming concept where a function solves a problem by calling **itself** to solve smaller, identical versions of the same problem.¹

Think of it like opening a set of Russian Matryoshka nesting dolls.²

1. You open the first doll.
2. Inside, you find a slightly smaller, *identical* doll.
3. You repeat the *same* action (opening the doll) on the new doll.³
4. You keep doing this until you reach the smallest, solid doll that can't be opened.⁴

This "smallest doll" is the most important part of recursion.

The Two Key Parts of Recursion

For a recursive function to work, it **must** have two parts:

1. **The Base Case:** This is the **stopping condition**.⁵ It's the simplest version of the problem that can be solved directly, without calling itself again. In the doll analogy, it's the smallest, solid doll.⁶ If you forget the base case, you get an **infinite recursion**, which will crash your program (a "stack overflow").⁷
 2. **The Recursive Step:** This is where the function calls itself, but with a *smaller* or *simpler* version of the problem. This step must move the problem closer to the base case.⁸
-

A Simple Example: Countdown

Here's a simple recursive function in Python that counts down from a number.

Python

```
def countdown(n):
```

```
    # 1. The Base Case (the stopping condition)
```

```
    if n < 0:
```

```
        return # Stop the function
```

```
    # The "work"
```

```
    print(n)
```

```
    # 2. The Recursive Step (calling itself with a smaller problem)
```

```
    countdown(n - 1)
```

```
# Let's run it
```

```
countdown(3)
```

What Happens:

1. `countdown(3)` runs. It's not < 0 , so it prints 3 and calls `countdown(2)`.
2. `countdown(2)` runs. It's not < 0 , so it prints 2 and calls `countdown(1)`.
3. `countdown(1)` runs. It's not < 0 , so it prints 1 and calls `countdown(0)`.
4. `countdown(0)` runs. It's not < 0 , so it prints 0 and calls `countdown(-1)`.

5. `countdown(-1)` runs. It **is** < 0 , so it hits the **base case** and just returns. The chain is broken, and the function stops.

Output:

3
2
1
0

Q19. Explain the drawback of recursion?

The two primary drawbacks of recursion, which I always weigh during design, are:

- **Risk of Stack Overflow:** This is the most significant. Each recursive call adds a new frame to the call stack, which consumes finite memory. If the recursion is too deep—either from a flawed base case or simply a very large input—the stack can run out of memory. This results in a "stack overflow error" and crashes the program.
- **Performance Overhead:** Recursion is often less efficient than an equivalent iterative (loop-based) solution. The repeated function calls are not "free." They create overhead in terms of both speed (the time it takes to push/pop from the stack) and memory (the space on the stack itself). This overhead can become a performance bottleneck.

Additionally, recursive code can sometimes be harder to **debug**. Tracing the flow of execution as it moves down and back up the call stack can be less intuitive than stepping through a simple loop.

This is why, for simple tasks, a loop is almost always preferred. I reserve recursion for problems that are naturally recursive, like navigating a tree structure, where its clarity can outweigh these drawbacks.

Q20. What is Memoization?

Memoization is an **optimization technique** where you store the results of expensive function calls and return the cached result when the same inputs are used again.

Think of it as a "lookup notebook" for a function.

How It Works

1. **First Call:** The function is called with a new input (e.g., `calculate(5)`). It performs the full, "expensive" calculation, gets the result, and then **stores it in a cache** (like a dictionary) with the input as the key: `{'5': result}`.

2. **Second Call:** The function is called again with the *same* input (calculate(5)). This time, it first checks its notebook. It sees it already has the result for "5," so it **returns the stored result immediately** instead of doing the calculation all over again.

A Classic Example

The best example is a recursive **Fibonacci sequence**. A function fib(n) calls fib(n-1) and fib(n-2).

- To calculate fib(5), it needs fib(4) and fib(3).
- To calculate fib(4), it needs fib(3) and fib(2).

Notice that fib(3) is being calculated twice, from scratch. Without memoization, calculating fib(20) would result in fib(2) being calculated over 2,500 times.

With memoization, fib(2) is calculated **once**. Every subsequent request for fib(2) just pulls the answer from the cache, turning an exponential problem into a linear one.

Key Benefit

In short, memoization is a **trade-off**. It uses slightly more **memory** (to store the cache) to gain a massive improvement in **speed**.

Q21. Difference between Filter and Map?

Feature	map	filter
Purpose	To transform each item in a list.	To select items from a list.
Output Size	Returns a new list of the exact same size as the original.	Returns a new list of the same size or smaller .
Function's Role	The function you provide returns the new, transformed value .	The function you provide returns a boolean (True/False) to keep or discard the item.
Content of Result	The new list contains the transformed items .	The new list contains a subset of the original, unchanged items .
Analogy	A factory assembly line (changes every item).	A quality control checkpoint (selects only some items).

Q22. What is Object-oriented programming?

Q23. What are features of object oriented programming?

Object-Oriented Programming (OOP) is a way of coding that bundles **data** (like color) and the **functions** that use it (like drive()) into single units called **objects**.

You create objects using a **Class**, which is like a blueprint. For example, a Car blueprint defines that all cars will have a color and a drive() method. The actual Car object is the thing you build from that blueprint.

OOP is built on four main ideas/ features:

1. **Encapsulation:** Hiding an object's internal complexity. (You use the steering wheel, not the raw mechanics.)
2. **Abstraction:** Hiding *how* something works. (You just press the pedal to accelerate(); the complex logic is hidden.)
3. **Inheritance:** Reusing code by letting one class (a SportsCar) get all the features from another (a Car).
4. **Polymorphism:** Letting different objects respond to the same command in their own way (a Car and a Boat both have a move() method, but they execute it differently).
- 5.

Q24. What is Difference between abstraction & encapsulation?

The main difference:

- **Abstraction** is about **hiding complexity**. It shows you *what* something does, but not *how* it does it. It's a design-level concept.
- **Encapsulation** is about **hiding data**. It bundles data and the methods that operate on that data into a single unit (an object) to protect it from outside interference. It's an implementation-level concept.

Simple Analogy: A Car 🚗

- **Abstraction:** The steering wheel, pedals, and gear stick.
 - You just press the "accelerate" pedal. You don't need to know the complex details of fuel injection, combustion, or the drivetrain. The *how* is hidden, leaving you with a simple interface.
- **Encapsulation:** The sealed engine block.
 - The engine's pistons, valves, and crankshaft are all *bundled* together and *hidden* inside a protective casing. You can't access them directly. You can only interact

with them through the car's public interface (like pressing the accelerator, which *then* tells the engine what to do).

Q25. What are the access modifiers in python?

Python doesn't have true access modifiers like private or protected. It uses **naming conventions** as hints:

1. **Public** (variable):
 - The default. Meant to be used by anyone.
2. **Protected** (`_variable`):
 - A single underscore is a **hint** that this is for internal use within the class or its subclasses. It's not enforced.
3. **Private** (`__variable`):
 - A double underscore triggers **name mangling** (renames the variable to `_ClassName__variable`). This isn't for security, but to **prevent accidental name collisions** in child classes.

Q26. What is constructor?

A **constructor** is a special method in a class that is **automatically called** when you create a new instance (or "object") of that class.¹

Its main purpose is to **initialize** the object, setting the starting values for its attributes (the data it holds).²

Analogy: A Car Blueprint 🚗

Think of a Class as a **blueprint** for a car. The constructor is the **assembly line** that actually builds the car. When you create a new car object, the constructor runs.³ It takes in the initial specs (like color and model) and assigns them to the new car, ensuring it's in a valid state from the moment it's created.

Example in Python

In Python, the constructor method is always named `__init__`.

```
class Car:
    # This is the constructor
    def __init__(self, make, model, year):
        print("Building the car...")
```

```

        # Assigning initial values to the object's attributes

        self.make = make

        self.model = model

        self.year = year


# This line automatically calls the __init__ constructor:
my_mustang = Car("Ford", "Mustang", 1969)


# The object is now fully initialized:

print(my_mustang.make) # Output: Ford
print(my_mustang.year) # Output: 1969

```

When Car(...) is called, Python creates an empty object and then immediately calls the `__init__` method on it, passing the new object as the first self argument.

Q27. What is destructor?

A **destructor** is a special method that is **automatically called** right before an object is destroyed and its memory is reclaimed.¹

Its main purpose is to **"clean up"** any resources the object was using.² This could include closing an open file, disconnecting from a database, or releasing a network connection, preventing memory leaks.³

Simple Python Example

In Python, the destructor method is named `__del__`.

```

class FileHandler:

    # Constructor: Runs when the object is created

    def __init__(self, filename):

        print(f"Opening {filename}...")

        self.filename = filename

```

```

# Destructor: Runs when the object is about to be destroyed

def __del__(self):
    print(f"Cleaning up and closing {self.filename}...")

# 1. Create the object (Constructor __init__ is called)
my_file = FileHandler("document.txt")

# 2. When the program is done with the object,
# we can delete it (or it will be deleted automatically)
print("About to delete the object...")

del my_file

```

Output:

Opening document.txt...

About to delete the object...

Cleaning up and closing document.txt...

Q28. What is Difference between Method and Constructure

A **constructor** is a special method that **initializes** an object *when it's created*.¹ It runs **automatically**.²

A **method** is a regular function inside a class that defines an object's **behavior**.³ It must be **called manually**.

Comparison Table

Feature	Constructor	Method
Purpose	To initialize the object's starting state (e.g., set attributes).	To perform an action or define the object's behavior.
Execution	Called automatically once, when you create the object.	Called manually by name, whenever you need it.

Name	Has a specific, reserved name (e.g., <code>__init__</code> in Python).	Can have any valid name (e.g., <code>drive()</code> , <code>get_name()</code>).
Return Value	Its job is to set up the object. It doesn't explicitly return a value.	Can return any type of value (a number, string, list, etc.).

Q29. Difference Between class variable & instance variable?

An **instance variable** is unique to each object (instance) and holds its own personal data.

A **class variable** is shared by *all* objects of that class. If you change it, it changes for every instance.

Feature	Instance Variable	Class Variable
Owner	Belongs to an individual object .	Belongs to the class itself .
Scope	Unique to each object.	Shared by all objects.
Definition	Defined inside the <code>__init__</code> constructor (using <code>self</code>).	Defined outside all methods, directly under the class.
Purpose	For data unique to an object (e.g., name, color).	For data shared by all objects (e.g., species, number_of_wheels).
Analogy	A person's unique name .	The species of all humans ("Homo sapiens").

Q30. Difference Between local variable & global variable?

Feature	Local Variable	Global Variable
Scope	Accessible only inside the function where it's defined.	Accessible from anywhere in the program (global scope).
Definition	Defined inside a function or code block.	Defined outside of all functions, at the top level.
Lifetime	Created when the function is called and destroyed when it ends.	Created when the program starts and destroyed when it ends.
Analogy	A private notebook for one person's use.	A public whiteboard for everyone in the office to see.
Best Use	For temporary data needed only for that one task. (Safer)	For constants or values that <i>must</i> be shared. (Riskier)

Q31. Difference Between formal parameters & actual parameters?

Feature	Formal Parameters	Actual Parameters
Definition	The variables listed in a function's definition .	The real values or expressions passed to a function when it's called .
Location	In the function signature (the def line).	In the function call (the line that runs the function).
Analogy	Placeholders or empty parking spots (e.g., spot_1, spot_2).	Actual values or the cars that fill the spots (e.g., a 'Ford', a 'Honda').
Common Name	Often just called Parameters .	Often called Arguments .
Purpose	They define <i>what</i> kind of data the function <i>expects</i> .	They provide the <i>actual</i> data the function <i>will work with</i> .

Q32. Difference Between keyword argument & required argument?

This distinction is about *how* arguments are defined and *how* they are passed.

- A **Required Argument** is a parameter in the function definition that **has no default value**. You *must* provide a value for it when you call the function.
- A **Keyword Argument** is a name=value pair used when you **call** the function. This lets you pass arguments *out of order* and is often used for optional parameters.

The opposite of a **Required Argument** is an **Optional Argument** (one with a default value).
The opposite of a **Keyword Argument** is a **Positional Argument** (where the order matters).

Feature	Required Argument (Parameter)	Keyword Argument (Call)
Location	In the function definition (def).	In the function call (when you run it).
Purpose	Defines a value that the function must receive.	Provides a value by name , so order doesn't matter.
Syntax	def my_func(name): (name has no default)	my_func(name="Alice") (uses name=value)
Mandatory?	Yes . You must provide a value for it.	Often optional , as it's typically used for parameters that have a default value.
Analogy	A required field on a form (like "Your Name").	A labeled field on a form (like "Last Name: Smith") that you can fill in any order.

Q33. What is variable length argument?

A **variable-length argument** is a feature that allows a function to accept an **indefinite number of arguments**.

Instead of defining a function with a fixed number of parameters (like `def my_func(a, b)`), you can use a special syntax to "pack" all extra arguments into a single variable.

In Python , there are two types:

1. ***args (Tuple)**

This lets a function accept any number of **positional arguments** (ones without names). They are collected into a **tuple**.

```
def print_all(*args):  
    print(args)  
  
print_all(1, "hello", True)  
# Output: (1, 'hello', True)
```

2. ****kwargs (Dictionary)**

This lets a function accept any number of **keyword arguments** (ones *with* names). They are collected into a **dictionary**.

```
def print_info(**kwargs):  
    print(kwargs)  
  
print_info(name="Alice", age=30)  
# Output: {'name': 'Alice', 'age': 30}
```

Q34. What is class & object?

A **class** is like a **blueprint**. For example, a blueprint for a car. It defines all the properties (like color or number of doors) and the things it can do (like `start_engine()` or `drive()`). But the blueprint itself isn't a car; it's just the plan.

An **object** is the **actual car** you build *from* that blueprint. It's a real, specific thing.

So, you can use that one "Car" blueprint (the class) to build many different, actual cars (the objects), like a red Ford Mustang or a blue Tesla Model S. Each is a separate object, but they both follow the same plan from the class.

Q35. What is self-parameter?

The self parameter is a special variable used inside a class's methods that refers to the **object itself** (the instance of the class).

Its main purpose is to give the object's methods a way to access and modify its *own* unique attributes and other methods.

Simple Analogy: "My"

Think of self as the word "**my**" or "**me**."

If you have a Car blueprint (the class) and you build two *actual* cars (the objects), car1 and car2, self is how they tell themselves apart.

When you tell car1 to drive(), the drive() method uses self to say, "I need to get *my* color" (self.color) or "I need to check *my* speed" (self.speed). It ensures car1 doesn't accidentally get car2's color.